

Sri Lanka Institute of Information Technology



IE3132 – DevOps-Security

Year 3, Semester 1- 2026

Assignment 01

IT Number	Student Name
IT24101491	Pethvan p p
IT24102268	Aluthge M.N
IT24101916	Adithya E.A.D.D.D
IT24101727	Shanthoshika S

Table of Contents

Executive Summary	- 3 -
Application, Architecture and Containerizations	- 4 -
Container hardening and principle of least privilege.....	- 4 -
Network segmentation and trust boundary enforcement	- 4 -
Threat modeling and risk assessment	- 6 -
Stride threat model.....	- 6 -
Risk evaluation matrix.....	- 7 -
Threat to control mapping.....	- 7 -
Secure Coding: Exploit-and-Fix Walkthroughs	- 9 -
NoSQL Injection (Authentication Bypass)	- 9 -
Stored Cross-Site Scripting (XSS)	- 12 -
Insecure Direct Object Reference (IDOR)	- 14 -
Insecure Cryptographic Storage (Plaintext Passwords)	- 16 -
DevSecOps CI/CD Pipeline & Security Automation	- 20 -
GitHub Actions Workflow:.....	- 20 -
Secrets Management Approach	- 22 -
Industry Trends and Case Study Analysis	- 23 -
The Hidden Risks of Open Source: Lessons from the event-stream Supply Chain Attack	- 23 -
Defending Our Project: How Our CI/CD Pipeline Stops Supply Chain Attacks	- 24 -
Project reflection and future improvements	- 26 -
Individual Contribution Statement	- 27 -
AI usage disclosure	- 28 -
Reference	- 29 -

Executive Summary

This report presents a comprehensive DevOps security assessment and remediation lifecycle conducted on the OWASP NodeGoat application. The objective is to shift security practice left by identifying critical application-layer vulnerabilities and establishing automated policy enforcing CI/CD pipelines through controlled exploitations, threat analysis and automated security control

Key technical achievements

- **Vulnerability identification and remediation**
 - NoSql Injection
 - Stored cross-site scripting
 - Insecure direct object reference
 - Insecure cryptography storage
 - Static application security testing
- **Continuous security automation**
 - Secret scanning
 - SAST
 - Software composition analysis
 - Container security
- **Enterprise secrets management**
- **Software supply chain risk analysis**

Application, Architecture and Containerizations

Multi-container Stack

The deployment architecture partitions OWASP NodeGoat into two discrete containerized tiers

- The application runtime tier(nodegoat-web)
- Data storage tier(nodegoat-db)

Using docker compose the system build and start in fully self container offline status single command **docker compose up --build**. The web container manage express application handling and isolated mongo-db database management manage in db docker using tcp port 27017 for monitoring scripts and logs

Container hardening and principle of least privilege

To mitigate container breakout and privilege escalation risks the application container explicitly abandons default execution the Docker file apply user mode node ensuring all child application process execution under and unprivileged user space

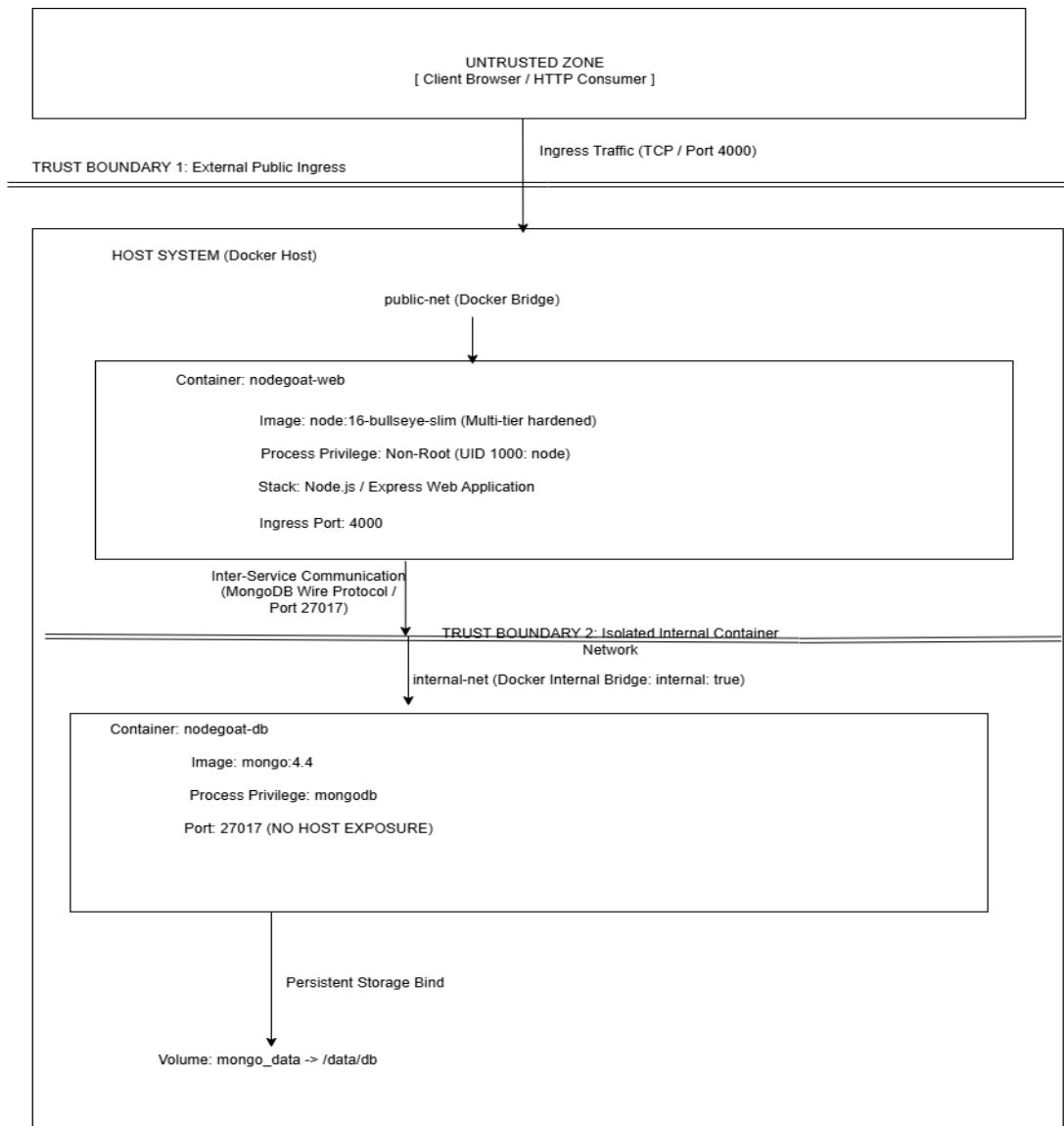
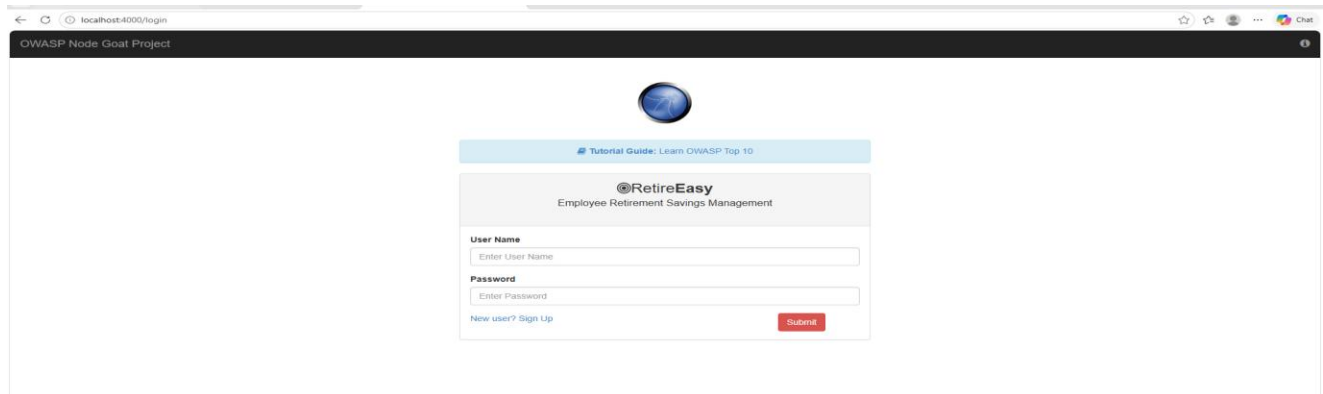
Network segmentation and trust boundary enforcement

Network boundaries are made with docker network bridge

- **public network**
act as the ingress channel bridging host port 4000 to the web container http traffic
- **internal network**
configure with **internal:true** to prevent external inbound or outbound routing port 27017 to only accessible inside net to ensure database cannot reach host machine or public network \

```
E:\sliit\devops\devsecops-assignment>docker compose -f docker\docker-compose.yml ps
NAME                IMAGE              COMMAND                  SERVICE    CREATED        STATUS          PORTS
nodegoat-db         mongo:4.4          "docker-entrypoint.s..." database    59 minutes ago Up About a minute 27017/tcp
nodegoat-web        sha256:6ffe9939936472a9b403958905d4d3f5121842299e1b64c8e0b25c36c75bfa2 "docker-entrypoint.s..." web        59 minutes ago Up About a minute 0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp
E:\sliit\devops\devsecops-assignment>
```

```
E:\sliit\devops\devsecops-assignment>docker exec nodegoat-web whoami
node
```



Threat modeling and risk assessment

Stride threat model

Below are four realistic, application-specific threats that might target the NodeGoat architecture. The threats were identified using the STRIDE methodology.

- **Spoofing (Authentication Bypass via NoSQL Injection):** An attacker can launch a NoSQL injection attack by sending a malicious JSON object to the /login API. By replacing the password value with a MongoDB operator, such as {"password": {"\$ne": null}}, and taking advantage of the fact that Express does not enforce typechecking on the input, the attacker can bypass authentication and hijack an administrator's session. No credentials would be required to perform this attack, apart from basic knowledge of the MongoDB query language.
- **Tampering (Insecure Direct Object Reference on Allocations):** An authenticated attacker can tamper with allocation records by modifying the request sent to the server when the attacker attempts to update their asset allocation. By changing the userId value in the request to another value, the attacker can effectively change the allocation records of another user. The vulnerability stems from the fact that the server does not check whether the userId belongs to the authenticated attacker.
- **Information Disclosure (Hardcoded Database Credentials):** A developer could accidentally commit the database credentials required to connect to the MongoDB server and sign cookies for the Express framework in the config/env/development.js file or the docker-compose.yml file. If this information becomes available to an attacker, they can utilize it to connect to the database directly and forge valid session cookies to hijack the administrator's session.
- **Denial of Service (Unbounded Payload Exhaustion):** An attacker can utilize a large JSON payload to exhaust the CPU resources on the Node.js server. Since Node.js runs on a single thread, any request that requires deep traversal of the JSON object will monopolize the CPU and deny service to other users. This threat is particularly concerning because there is no built-in defense against it in the Node.js framework itself.

Risk evaluation matrix

Each threat is scored on a 5x5 matrix evaluating **likelihood**(how easy/often the attack can be executed) and **impact** resulting in an overall risk rating

threat	likelihood	impact	Overall risks
Spoofing(login no SQLi)	5	5	25
Tampering (allocation IDOR)	4	4	16
Info disclosure(secrets in Git)	5	4	20
Dos(Payload exhaustion)	3	3	9

Threat to control mapping

This table directly identified threats to the specific security controls implemented in pipeline and codebase

Threat category	Vulnerability scenario	Risk level	Mitigating control	Codebase/pipeline location
Spoofing	NoSQL injection in login bypassing authentication	Critical	Input Validation & Sanitization: Implement strict allow-list type checking (typeof password === 'string') before querying the database.	app/routes/session.js and app/data/user-dao.js
Tampering	Modifying other users' allocations via IDOR	High	Server-Side Authorization: Ignore client-provided user IDs and enforce lookups strictly using the verified req.session.userId	app/routes/allocations.js

Information disclosure	Hardcoded secrets committed to source control	High	Secret Scanning & Environment Injection: Gitleaks prevents secret commits, and GitHub Actions Encrypted Secrets inject credentials at runtime	.github/workflows/devseco ps-pipeline.yml
Denial of service	Event-loop blocking via massive JSON payloads	Medium	SAST & Infrastructure Hardening: Semgrep flags missing limits; express.json({ limit: '10kb' }) restricts body size.	.github/workflows/devseco ps-pipeline.yml

Secure Coding: Exploit-and-Fix Walkthroughs

NoSQL Injection (Authentication Bypass)

Vulnerability

The image shows two screenshots of the OWASP Node Goat Project application. The top screenshot is the login page, which features a header with the project name, a logo, a tutorial guide link, and a login form for 'RetireEasy' with fields for 'User Name' and 'Password'. The bottom screenshot shows the 'Benefits' page, which includes a sidebar with 'Benefits' and 'Logout' links, and a table for managing employee benefits.

RetireEasy Login Form

©RetireEasy
Employee Retirement Savings Management

User Name
Enter User Name

Password
Enter Password

[New user? Sign Up](#)

Benefits Page

Benefits Start Date

Employee ID	First Name	Last Name	Benefits Start Date
2	John	Doe	01/10/2030 Save
3	Will	Smith	11/30/2025 Save

app/data/user-dao.js

```
this.validateLogin = (userName, password, callback) => {  
  
  usersCol.findOne({  
    userName: userName,  
    password: password  
  }, (err, user) => {  
    if (err) return callback(err, null);  
  
    if (user) {  
      callback(null, user); // Logs the user in!  
    } else {  
      const invalidPasswordError = new Error("Invalid password");  
      invalidPasswordError.invalidPassword = true;  
      callback(invalidPasswordError, null);  
    }  
  });  
};
```

Burp Suite Community Edition v2026.8 - Temporary Project				
Dashboard	Target	Proxy	Intruder	Repeater
CollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscover				
Intercept	HTTP history	WebSockets history	Match and replace	Proxy settings
Intercept onForwardDrop				
Request to http://localhost:40				
Time	Type	Direction	Method	URL
18:42:50 23 Sep ...	HTTP	→ Request	POST	http://localhost4000/login

Request

PrettyRawHex

```
5 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"  
6 sec-ch-ua-mobile: 70  
7 sec-ch-ua-platform: "Windows"  
8 Accept-Language: en-US,en;q=0.9  
9 Upgrade-Insecure-Requests: 1  
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36  
11 Origin: http://localhost:4000  
12 Content-Type: application/x-www-form-urlencoded  
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7  
14 Sec-Fetch-Site: same-origin  
15 Sec-Fetch-Mode: navigate  
16 Sec-Fetch-User: ?1  
17 Sec-Fetch-Dest: document  
18 Referer: http://localhost:4000/login  
19 Accept-Encoding: gzip, deflate, br  
20 Cookie: connect.sid=s%3A19vlnK-ln47DcHDJLTquVtEdcA2DqdJx.34HuV2Bhp46oaeqvAlstqqghkR42KwHeoLidWVCMeIIY  
21 If-None-Match: W/"id52-M5Uq2DeVQcHW/nBvV7BBpBx2npo"  
22 Connection: keep-alive  
23  
24 {  
25   "username": "admin",  
26   "password": ({ "ne": null })  
27 }
```

0 highlights

Fix

app/data/user-dao.js

```

this.validateLogin = (userName, password, callback) => {

  const comparePassword = (fromDB, fromUser) => {
    return fromDB === fromUser;
  };

  const validateUserDoc = (err, user) => {

    if (err) return callback(err, null);

    if (user) {
      if (comparePassword(password, user.password)) {
        callback(null, user);
      } else {
        const invalidPasswordError = new Error("Invalid password");
        invalidPasswordError.invalidPassword = true;
        callback(invalidPasswordError, null);
      }
    } else {
      const noSuchUserError = new Error("User: " + user + " does not exist");
      noSuchUserError.noSuchUser = true;
      callback(noSuchUserError, null);
    }
  };
};

```

OWASP Node Goat Project



[Tutorial Guide: Learn OWASP Top 10](#)

©RetireEasy

Employee Retirement Savings Management

Invalid username



User Name

Enter User Name

Password

Enter Password

[New user? Sign Up](#)

Submit

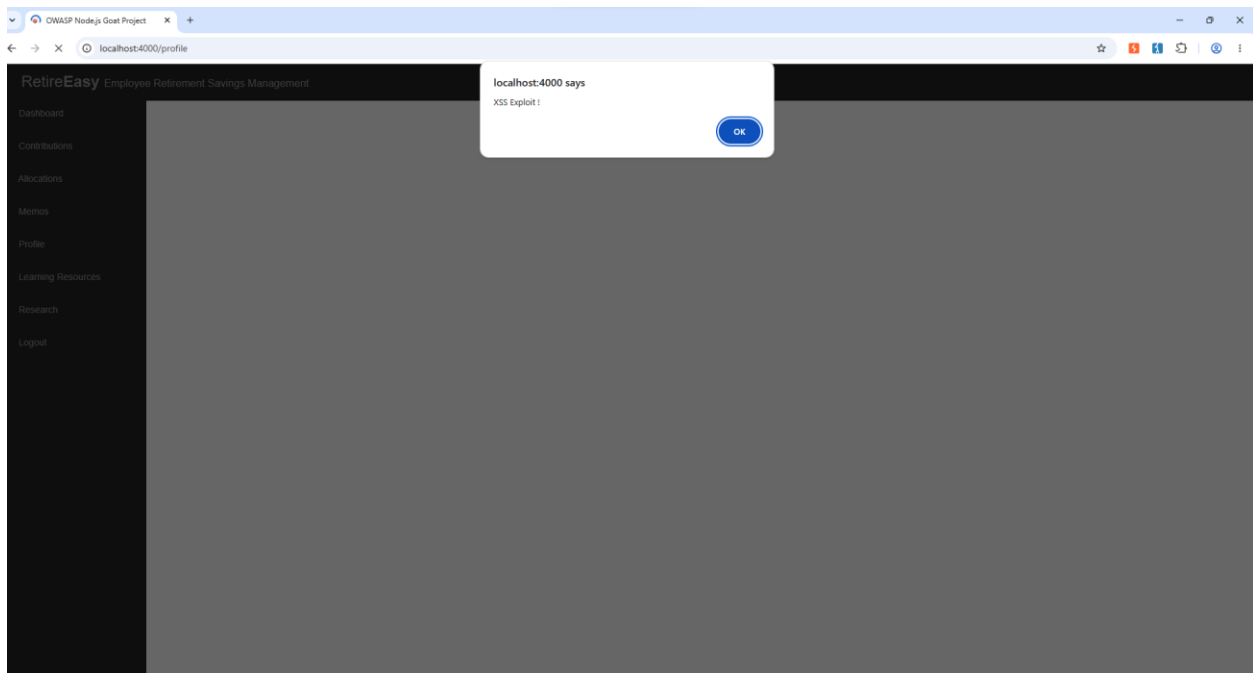
Stored Cross-Site Scripting (XSS)

Vulnerability

The screenshot shows a web browser window with the URL `localhost:4000/profile`. The page is titled "RetireEasy Employee Retirement Savings Management". On the left is a dark sidebar with navigation links: Dashboard, Contributions, Allocations, Memos, Profile, Learning Resources, Research, and Logout. The main content area is titled "My Profile" and contains an "Edit Profile" form. The form fields are as follows:

- First Name:** A text input field containing the payload `<script>alert('XSS Exploit!')</script>`.
- Last Name:** A text input field containing the value "Doe".
- SSN:** A text input field with the placeholder "Enter SSN".
- Date of Birth:** A date picker field with the placeholder "mm/dd/yyyy".
- Bank Account #:** A text input field containing the value "1231435#".
- Bank Routing #:** A text input field containing the value "1231435#". Below this field is a note: "Must be entered as digits with a suffix of #. For example: 0190212#".
- Address:** A text input field with the placeholder "Enter address".
- Website:** A text input field with the placeholder "https://".

At the bottom of the form is a "Submit" button and a link that says "Google search this profile by name".



app/routes/profile.js

```
    this.handleProfileUpdate = (req, res, next) => {
      const {
        firstName,
        lastName,
        ssn,
        dob,
        address,
        bankAcc,
        bankRouting,
      } = req.body;
      const regexPattern = /^[0-9]{4}-[0-9]{4}-[0-9]{4}$/;
      const testComplyWithRequirements = regexPattern.test(bankRouting);
      if (testComplyWithRequirements !== true) {
        const firstNameSafeString = firstName;
        return res.render("profile", {
          updateError: "Bank Routing number does not comply with requirements for format specified",
          firstNameSafeString,
          lastName,
          ssn,
          dob,
          address,
          bankAcc,
          bankRouting,
          environmentalScripts
        });
      }

      const {
        userId
      } = req.session;

      profile.updateUser(
        parseInt(userId),
        firstName,
        lastName,
        ssn,
        dob,
        address,
        bankAcc,
        bankRouting,
        (err, user) => {
          if (err) return next(err);
          user.updateSuccess = true;
          user.userId = userId;

          return res.render("profile", {
            ...user,
            environmentalScripts
          });
        }
      );
    };
  };
};

module.exports = ProfileHandler;

```

Fix

RetireEasy Employee Retirement Savings Management

My Profile

Profile updated successfully

Edit Profile

First Name

Last Name

DOB

SSN

Date of Birth

Bank Account #

Bank Routing #

Address

Website

Submit

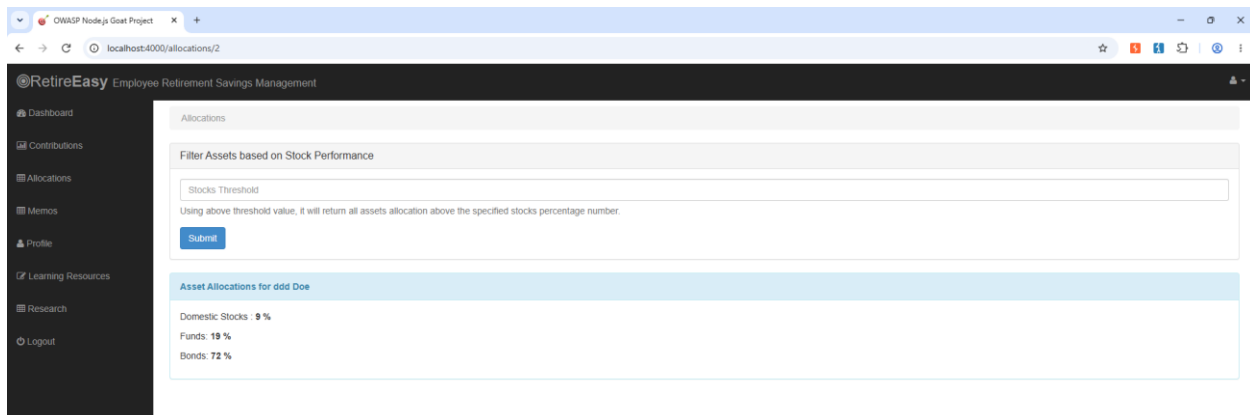
Must be entered as digits with a suffix of #. For example: 0198210#

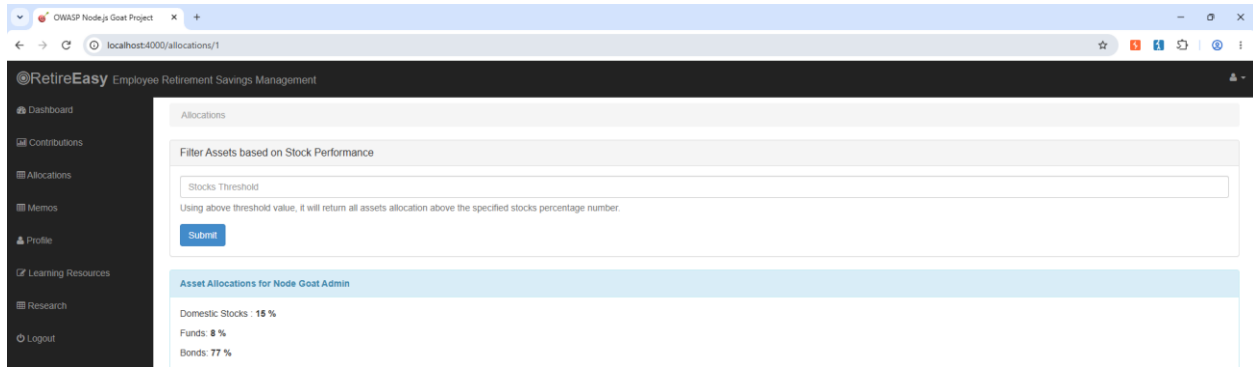
app/routes/profile.js

```
10 this.handleProfileUpdate = (req, res, next) => {
11   const escapeHtml = (unsafe) => {
12     if (!unsafe) return "";
13     return String(unsafe)
14       .replace(/&/g, "&amp;")
15       .replace(/</g, "&lt;")
16       .replace(/>/g, "&gt;")
17       .replace(/"/g, "&quot;")
18       .replace(/'/g, "&#039;");
19   };
20
21   const firstName = escapeHtml(req.body.firstName);
22   const lastName = escapeHtml(req.body.lastName);
23   const ssn = escapeHtml(req.body.ssn);
24   const dob = escapeHtml(req.body.dob);
25   const address = escapeHtml(req.body.address);
26   const bankAcc = escapeHtml(req.body.bankAcc);
27   const bankRouting = req.body.bankRouting;
28
29   const regexPattern = /^[0-9]*$/;
30   const testComplyWithRequirements = regexPattern.test(bankRouting);
31
32   if (testComplyWithRequirements !== true) {
33     const firstNameSafeString = firstName;
34     return res.render("profile", {
35       updateError: "Bank Routing number does not comply with requirements for format specified",
36       firstNameSafeString,
37       lastName,
38       ssn,
39       dob,
40       address,
41       bankAcc,
42       bankRouting: escapeHtml(bankRouting),
43       environmentalScripts
44     });
45   }
46
47   const { userId } = req.session;
48
49   profile.updateUser(
50     parseInt(userId),
51     firstName,
52     lastName,
53     ssn,
54     dob,
55     address,
56     bankAcc,
57     escapeHtml(bankRouting), // Sanitize before saving to DB
58     (err, user) => {
59       if (err) return next(err);
60
61       user.updateSuccess = true;
62       user.userId = userId;
63
64       return res.render("profile", {
65         ...user,
66         environmentalScripts
67       });
68     }
69   );
70 }
71
72 module.exports = ProfileHandler;
```

Insecure Direct Object Reference (IDOR)

Vulnerabilities





app/routes/allocations.js

```

app > app > routes > JS allocations.js > ...
1  const AllocationsDAO = require("../data/allocations-dao").AllocationsDAO;
2  const {
3    environmentalScripts
4  } = require("../config/config");
5
6  function AllocationsHandler(db) {
7    "use strict";
8
9    const allocationsDAO = new AllocationsDAO(db);
10
11    this.displayAllocations = (req, res, next) => {
12      /*
13       // Fix for A4 Insecure DOR - take user id from session instead of from URL param
14       const { userId } = req.session;
15       */
16      const {
17        userId
18      } = req.params;
19      const {
20        threshold
21      } = req.query;
22
23      allocationsDAO.getByUserIdAndThreshold(userId, threshold, (err, allocations) => {
24        if (err) return next(err);
25        return res.render("allocations", {
26          userId,
27          allocations,
28          environmentalScripts
29        });
30      });
31    };
32  }
33
34  module.exports = AllocationsHandler;
35

```

Fix



app/routes/allocations.js

```
app > app > routes > JS allocations.js > AllocationsHandler > displayAllocations
1  const AllocationsDAO = require("../data/allocations-dao").AllocationsDAO;
2  const {
3    |   environmentalScripts
4  } = require("../config/config");
5
6  function AllocationsHandler(db) {
7    "use strict";
8
9    const allocationsDAO = new AllocationsDAO(db);
10
11    this.displayAllocations = (req, res, next) => {
12      |
13      const { userId } = req.session;
14      |
15      if (parseInt(req.params.userId, 10) !== userId) {
16        |   return res.status(403).send("HTTP 403 Forbidden: Access Denied. You can only view your own allocations.");
17      }
18      |
19      allocationsDAO.getByUserId(parseInt(userId), (err, allocations) => {
20        |   if (err) return next(err);
21        |   return res.render("allocations", { userId, allocations, environmentalScripts });
22      });
23    };
24  }
25
26  module.exports = AllocationsHandler;
27
```

Insecure Cryptographic Storage (Plaintext Passwords)

Vulnerable

```
P5 E:\slit\devops\devsecops-assignment\devsecops-year3-semester-1\devsecops-year3-semester-1> docker exec -it nodegoat-db mongo
MongoDB shell version v4.4.30
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("750bc387-c6dd-4cfa-9706-f91837b87cee") }
MongoDB server version: 4.4.30
---
The server generated these startup warnings when booting:
  2026-09-23T15:18:45.271+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2026-09-23T15:18:45.505+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-09-23T15:18:45.506+00:00: /sys/kernel/mm/transparent_hugepage/enabled is 'always'. We suggest setting it to 'never'
---
> use nodegoat
switched to db nodegoat
> db.users.find({userName: "admin"}).pretty()
{
  "_id" : 1,
  "userName" : "admin",
  "firstName" : "Node Goat",
  "lastName" : "Admin",
  "password" : "Admin_123",
  "isAdmin" : true
}
> |
```

app/data/user-dao.js

```

16
17   this.addUser = (userName, firstName, lastName, password, email, callback) => {
18
19       const user = {
20           userName,
21           firstName,
22           lastName,
23           benefitStartDate: this.getRandomFutureDate(),
24           password
25       };
26

```

```

    this.validateLogin = (userName, password, callback) => {

        const comparePassword = (fromDB, fromUser) => {
            return fromDB === fromUser;
        };

        const validateUserDoc = (err, user) => {

            if (err) return callback(err, null);

            if (user) {
                if (comparePassword(password, user.password)) {
                    callback(null, user);
                } else {
                    const invalidPasswordError = new Error("Invalid password");

                    invalidPasswordError.invalidPassword = true;
                    callback(invalidPasswordError, null);
                }
            } else {
                const noSuchUserError = new Error("User: " + user + " does not exist");

                noSuchUserError.noSuchUser = true;
                callback(noSuchUserError, null);
            }
        };

        usersCol.findOne({
            userName: userName
        }, validateUserDoc);
    };

```

Fix

```
>
> db.users.find({userName: "passuser2"}).pretty()
{
  "_id" : 4,
  "userName" : "passuser2",
  "firstName" : "dsds",
  "lastName" : "sddssd",
  "benefitStartDate" : "2045-02-12",
  "password" : "$2a$10$j7sXU7mdnLOa51W2xq1M70RNJcvM7RNRAGA0.y9J0ewW3fCYGbv/a"
}
```

app/data/user-dao.js

```
this.addUser = (userName, firstName, lastName, password, email, callback) => {

  // SECURITY FIX: Generate a salt and hash the plaintext password
  const salt = bcrypt.genSaltSync(10);
  const hashedPassword = bcrypt.hashSync(password, salt);

  const user = {
    userName,
    firstName,
    lastName,
    benefitStartDate: this.getRandomFutureDate(),
    password: hashedPassword
  };
};
```

```

this.validateLogin = (userName, password, callback) => {

  const comparePassword = (plainTextPassword, hashedPassword) => {
    return bcrypt.compareSync(plainTextPassword, hashedPassword);
  };

  const validateUserDoc = (err, user) => {

    if (err) return callback(err, null);

    if (user) {
      if (comparePassword(password, user.password)) {
        callback(null, user);
      } else {
        const invalidPasswordError = new Error("Invalid password");
        invalidPasswordError.invalidPassword = true;
        callback(invalidPasswordError, null);
      }
    } else {
      const noSuchUserError = new Error("User: " + userName + " does not exist");
      noSuchUserError.noSuchUser = true;
      callback(noSuchUserError, null);
    }
  };

  usersCol.findOne({
    userName: userName
  }, validateUserDoc);
};

```

DevSecOps CI/CD Pipeline & Security Automation

GitHub Actions Workflow:

Trigger the Failed Pipeline Run

The screenshot shows a failed GitHub Actions workflow run for the 'security-gates' job. The workflow is titled 'security-gates' and is marked as 'failed now in 1m 24s'. The left sidebar shows the 'Summary' tab selected, with a red status icon for the 'security-gates' job. The main content area lists the steps of the workflow:

- Set up job (2s)
- Checkout Repository (1s)
- 1. Secret Scanning Gate (Gitleaks) (5s)
- 2. SAST Scan Gate (Semgrep) (21s)
- 3. SCA / Dependency Scan Gate (npm audit) (9s)
- Build Application Container (19s)
- 4. Container Image Scan Gate (Trivy) (22s)

The '4. Container Image Scan Gate (Trivy)' step is expanded, showing a list of logs with a red error icon. The logs indicate that the process completed with exit code 1. Below the workflow steps, the 'Annotations' section shows 2 errors, 1 warning, and 1 notice. The first error is for the 'security-gates' job, stating 'Process completed with exit code 1.' The second error is for the 'security-gates' job, stating 'Process completed with exit code 1.' The warning is for the 'security-gates' job, stating 'Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, gitleaks/gitleaks-action@v2. For more information see: <https://github.blog/changelog/2022-09-26-github-actions-deprecating-node-20/>. Show more'. The notice is for the 'security-gates' job, stating 'The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>'.

Trigger the Successful Pipeline Run

The screenshot shows a successful GitHub Actions workflow run for the 'security-gates' job. The workflow is titled 'Update devsecops-pipeline.yml #16' and is marked as 'succeeded now in 1m 29s'. The left sidebar shows the 'Summary' tab selected, with a green status icon for the 'security-gates' job. The main content area lists the steps of the workflow:

- Set up job (2s)
- Checkout Repository (1s)
- 1. Secret Scanning Gate (Gitleaks) (4s)
- 2. SAST Scan Gate (Semgrep) (22s)
- 3. SCA / Dependency Scan Gate (npm audit) (8s)
- Build Application Container (23s)
- 4. Container Image Scan Gate (Trivy) (18s)
- Post 4. Container Image Scan Gate (Trivy) (6s)
- Post Checkout Repository (1s)
- Complete job (0s)

The 'Annotations' section shows 1 error, 1 warning, and 1 notice. The error is for the 'security-gates' job, stating 'Process completed with exit code 1.' The warning is for the 'security-gates' job, stating 'Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, gitleaks/gitleaks-action@v2. For more information see: <https://github.blog/changelog/2022-09-26-github-actions-deprecating-node-20/>. Show more'. The notice is for the 'security-gates' job, stating 'The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>'.

Secrets Management Approach

Secret Provisioning and Image Layer Security

By removing all hardcoded credentials like ZAP API keys and MongoDB connection strings found in the source code (development.js and test.js) and replacing them with environment-variables-references, it becomes possible to ensure that the production code does not contain any secrets. To ensure that Docker image layers do not contain any hardcoded credentials as well (as those could be extracted via docker history command), such credentials are removed from the Dockerfile and only provided during the application's runtime. In the case of the local development environment, the variables are defined in a .env file that is not tracked by Git. As for the CI/CD multibranch pipeline, the variables are set during the GitHub Actions execution via the GitHub Actions Encrypted Secrets feature.

```
1 ZAP_API_KEY="v9dn0balpqas1pcc281tn5ood1"
```

```
app > config > env > JS testjs > <unknown>
1 module.exports = {
2   // If you want to debug regression tests, you will need the following.
3   zapHostName: "192.168.56.20",
4   zapPort: "8080",
5   // Required from Zap 2.4.1. This key is set in Zap Options -> API _Api Key.
6   zapApiKey: process.env.ZAP_API_KEY,
7   zapApiFeedbackSpeed: 5000 // Milliseconds.
8 };
9
```

```
app > config > env > JS developmentjs > <unknown>
1 module.exports = {
2   // If you want to debug regression tests, you will need the following which is also in the test config:
3   zapHostName: "192.168.56.20",
4   zapPort: "8080",
5   // Required from Zap 2.4.1. This key is set in Zap Options -> API _Api Key.
6   zapApiKey: process.env.ZAP_API_KEY,
7   // Required if debugging security regression tests.
8   zapApiFeedbackSpeed: 5000, // Milliseconds.
9   environmentalScripts: [
10    // jshint -W101
11    '<script>document.write("<script src='http://" + (location.host || "localhost").split(":")[0] + ":35729/livereload.js"></" + "script>");</script>'
12    // jshint +W101
13  ]
14 };
15
```

Industry Trends and Case Study Analysis

The Hidden Risks of Open Source: Lessons from the event-stream Supply Chain Attack

Modern software development rarely involves writing an application from scratch. Instead, developers rely on open-source ecosystems, importing pre-built libraries to handle everything from database connections to formatting text. While this drastically speeds up development, it introduces a massive blind spot: the software supply chain. When you install a single package, you are often pulling in dozens of other "transitive" dependencies—code written by strangers that you have never reviewed, running with the same permissions as your own application.

Instead of trying to hack through an organization's firewalls, modern attackers simply compromise these trusted open-source building blocks. If they can poison a popular library, developers will willingly download the malware straight into their secure networks.

One of the most fascinating and destructive examples of this is the 2018 attack on the Node.js package event-stream. At the time, event-stream was a massively popular utility tool, downloaded millions of times a week and used in major projects. However, the original developer was burned out and maintaining the project for free in his spare time. An unknown user going by the handle "right9ctrl" offered to help. By submitting a series of helpful, legitimate code updates, "right9ctrl" gradually earned the original developer's trust and was eventually handed full administrative control of the project.

This wasn't a smash-and-grab hack; it was a patient social engineering operation. Once in control, the attacker added a new, obscure sub-dependency called flatmap-stream to the project. On GitHub, the code for flatmap-stream looked perfectly harmless. But the version pushed to the npm registry—the code that actually gets downloaded when developers run npm install—contained highly obfuscated, encrypted malware.

The payload was brilliant because it was incredibly specific. The malware was designed to stay completely dormant unless it detected it was being executed inside a very specific environment: the build process for an application called Copay, a popular cryptocurrency wallet. If it detected Copay's specific description string, it used that string as a decryption key to unlock its true payload. The malware then injected a backdoor into the Copay app designed to harvest users' Bitcoin private keys and silently send them to an external server. Because it was buried inside a dependency of a dependency, it bypassed manual code reviews entirely and sat undetected for months.

Defending Our Project: How Our CI/CD Pipeline Stops Supply Chain Attacks

The event-stream incident proves that simply writing secure code is no longer enough; we have to secure the tools and libraries we build our code with. You cannot just assume a package is safe because it is popular. The DevSecOps pipeline we built for this project is specifically designed to catch and neutralize exactly this type of threat using a defense-in-depth approach.

➤ **Software Composition Analysis (npm audit)**

The first line of defense in our GitHub Actions workflow is the Software Composition Analysis (SCA) gate, powered by npm audit. Supply chain attacks rely on the fact that package managers automatically pull down the newest versions of dependencies. When a malicious package is finally discovered by security researchers, it is flagged and added to global vulnerability databases.

By running `npm audit --audit-level=high` in our pipeline, we are essentially running an automated background check on every single piece of third-party code in our project. The tool cross-references our `package-lock.json` file against these global databases. If a developer accidentally pulls in a compromised library like `flatmap-stream`, the SCA scan immediately detects the known CVE (Common Vulnerabilities and Exposures) signature. The pipeline exits with a failed status, turns red, and physically blocks the compromised code from moving forward in the build process. It acts as an automated guardrail against trusting poisoned open-source libraries.

➤ **Container Image Scanning (Trivy)**

While npm audit is great for checking Node.js libraries, it only looks at the application source code. It is entirely blind to the operating system it runs on. Attackers know this, which is why they frequently target OS-level packages (like the recent `xz-utils` backdoor in Linux).

This is where Trivy comes in as our second security gate. Trivy doesn't just look at the code; it scans the final, compiled Docker image (`pulina/nodegoat:latest`). This is the exact artifact that will be deployed to a live server. Trivy inspects both our application dependencies and the base Debian OS packages we use in our Dockerfile. By configuring our pipeline with `--exit-code 1` and targeting HIGH and CRITICAL vulnerabilities, we enforce a strict rule: if the container is vulnerable, the build fails. Even if a malicious package somehow sneaks past the npm audit step, Trivy will catch it when analyzing the final image, ensuring a poisoned container is never deployed to our users.

➤ **Software Bill of Materials (SBOM) for Incident Response**

The hardest part of defending against an attack like event-stream is the "zero-day" window. For several weeks, the flatmap-stream malware was completely unknown to the public. During that time, tools like npm audit and Trivy would not have flagged it because no CVE existed yet. When a zero-day drops and the news breaks, automated scanners cannot save you retroactively.

This is why generating a Software Bill of Materials (SBOM) is a critical industry trend. An SBOM is simply a comprehensive, machine-readable "ingredients list" of every single component inside your application, including all those deeply nested transitive dependencies.

If we wake up to news that a library in our ecosystem has been compromised, we don't have to spend days manually digging through our source code to see if we are vulnerable. We can simply search our SBOM for the malicious package name. It gives us instant visibility, allowing us to immediately determine if our application is at risk and roll out a patch in minutes rather than weeks.

By layering these automated tools—scanning the source dependencies, inspecting the compiled container, and maintaining total visibility into our software ingredients—our DevSecOps pipeline actively hardens the application against the realities of modern supply chain attacks.

Project reflection and future improvements

Project reflection :the value of “shifting left”

This project shows the critical value of "shifting. Automating security gates directly into ci/cd pipeline prove that security and development can be coexist. by automatically Catching vulnerabilities like NoSQL injection and hardcoded secrets at pull request stage preventing them from reaching production

Strategic future improvements

- **DAST Integration**
 - Integrate OWASP zap dynamically scan the running container for runtime misconfigurations and business logic flaws that SAST cannot detect
- **Automated SBOMs and artifact signing**
 - Utilize Syft to generated machine readable software bill during every build to cryptographically sign docker images
- **Dynamic secret vault**
 - Inject short lived automatically rotating credentials into the container
- **Policy as code**
 - To automatically scan IaC configures to ensure cloud deployments adhere to compliance rule before provisioning

Individual Contribution Statement

Member name	Student id	responsibilities	contribution
Pethvan P.P	IT24101491	Engineered the CI/CD pipeline (GitHub Actions), implemented automated security gates (Trivy, Semgrep, Gitleaks), and authored Section 4 (Pipeline) and Section 5 (Secrets Management)	25%
Aluthge M.N	IT24102268	Conducted the baseline threat modeling, performed manual vulnerability exploitation for NoSQL Injection and IDOR, and authored the corresponding exploit walkthroughs in Section 3.	25%
Shanthoshika S	IT24101727	Executed secure code remediation for Stored XSS and Insecure Cryptographic Storage (bcrypt), captured SAST comparison evidence, and authored the code patch walkthroughs in Section 3.	25%
Adithya E.A.D.D.D	IT24101916	Researched the event-stream supply chain attack, mapped project defenses (SCA/SBOMs) to industry trends, and authored Section 6 (Industry Case Study) and Section 7 (Reflection).	25%

AI usage disclosure

For this assignment we used Artificial Intelligence resources (Google Gemini/ChatGPT) to troubleshoot the GitHub Actions Workflow syntax, to check the document structure, and to review industry case studies. The technical materials generated as part of this project have not been automatically produced by any AI tools. All of the members of the group carried out the exploitation of the vulnerabilities, corrected the source codes, configured the CI/CD pipeline and performed the analysis.

Reference

- [1] OWASP Foundation, "OWASP NodeGoat Project," OWASP.org, 2026. [Online]. Available: <https://owasp.org/www-project-nodegoat/>.
- [2] T. Hunter II, "Compromised npm Package: event-stream," Medium, Nov. 26, 2018. [Online]. Available: <https://medium.com/intrinsic-blog/compromised-npm-package-event-stream-d47d08605502>[cite: 3].
- [3] CSIDE, "The Copay event-stream attack illustrates dependency risks," CSIDE Blog, Jul. 29, 2024. [Online]. Available: <https://cside.com/blog/the-copay-event-stream-attack-illustrates-dependency-risks>[cite: 2].
- [4] Reverera, "Compromised npm Package: event-stream and flatmap-stream packages lead to supply chain attack," Reverera Blog, Nov. 27, 2018. [Online]. Available: <https://www.reverera.com/blog/software-composition-analysis/compromised-npm-package-event-stream-and-flatmap-stream-packages-lead-to-supply-chain-attack/>[cite: 5].
- [5] National Institute of Standards and Technology (NIST), "Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities," NIST Special Publication 800-218, Feb. 2022. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-218/final>.
- [6] Aqua Security, "Trivy: Comprehensive Security Scanner," GitHub, 2026. [Online]. Available: <https://github.com/aquasecurity/trivy>.
- [7] Semgrep, "Semgrep: Lightweight static analysis for many languages," Semgrep.dev, 2026. [Online]. Available: <https://semgrep.dev/>.
- [8] Z. Rice, "Gitleaks: Protect and discover secrets using Gitleaks," GitHub, 2026. [Online]. Available: <https://github.com/gitleaks/gitleaks>.